

ENEE 467: Robotics Project Laboratory

Robot Arm Kinematics

Lab 2 - Part 2 (Hardware)

Learning Objectives

This part of the lab moves the kinematics tools you built in Part 1 onto the real UR3e arm. Working only with the Universal Robots Python interface, you will read the robot's live joint angles and reported tool pose and use them to validate your forward and inverse kinematics against the hardware. No motion commands are sent to the robot in this part. All interaction is read-only. After completing this part, you should be able to:

- Connect to a UR3e over the network and read its live joint and tool-pose state from Python
- Validate forward kinematics by comparing a model-based tool pose against the robot's reported pose
- Validate inverse kinematics by recovering joint configurations from a measured pose
- Account for the differences between an idealized kinematic model and a calibrated physical robot

Related Reading

1. K. Lynch and F. Park, *Modern Robotics: Mechanics, Planning, and Control*, Cambridge University Press, 2nd ed., 2019 (§4, §6).
2. Robotics Toolbox for Python: [GitHub](#).
3. ur_rtde Documentation: sdurobotics.gitlab.io/ur_rtde.
4. Universal Robots UR3e User Manual.

Lab Instructions

Follow the lab procedure closely. Some code blocks have been populated for you and some you will have to fill in yourself. Pay close attention to the lab **Questions** (highlighted with a box). Answer these as you proceed through the lab and include your answers in your lab writeup. There are several lab **Exercises** at the end of the lab manual. Complete the exercises during the lab and include your solutions in your lab writeup.

List of Questions

Question 1.	4
Question 2.	5
Question 3.	5
Question 4.	6

List of Exercises

Exercise 1. Forward-Kinematics Validation Across Configurations	6
Exercise 2. Inverse-Kinematics Round-Trip	6

Hardware Safety

Throughout this manual, boxes beginning with a **Question #** label contain questions you must answer as you progress through the lab:

Question 0. A question that you should answer as you work through the lab manual.

⚠ Safety and Setup Notices. Warnings highlight critical steps for hardware safety, teach-pendant operation, and common setup pitfalls. Always read these carefully before commanding motion on the UR3e.

⚠ Keep clear of the robot workspace when running autonomous motion. Confirm that the pendant is in Play mode before proceeding.

📖 Notes. These boxes provide background or theoretical context for the UR robot system. They are optional during lab time but useful for your report:

📖 UR3e kinematics are defined in a six-joint chain; MoveIt uses the URDF and joint limits to compute feasible IK solutions.

Code, commands, and output are formatted as follows:

```
ros2 launch ur_robot_driver ur_control.launch.py ...
```

```
[INF0] [launch]: UR3e driver successfully connected.
```

Files and directories appear in magenta teletype font (e.g., `~/lab7_ws/src`), while in-text code and variables use blue teletype (e.g., `object_pose`). Python type hints or XML tags appear in green (e.g., `str`, `<param>`).

1. Hardware Lab Procedure

⚠ Stay alert when working with the robot, and use the Emergency Stop Button on the Teach Pendant (see [Figure 1](#)) if needed. Always inform the team member(s) at the hardware station before reading from or commanding the robot. In this part the robot stays stationary. If it moves unexpectedly, press the emergency button immediately and contact your TA.

1.1 Prerequisites

This part assumes that you have completed Part 1 and are comfortable computing forward and inverse kinematics for the UR3 with the Robotics Toolbox for Python. Familiarity with the Bash shell and Docker, at the level used in Part 1, is also assumed.

1.2 Getting the Lab Code

The hardware procedure lives on a separate branch of the lab-2 repository. From your host system, open a terminal, navigate to the `~/Labs` directory, and clone the hardware branch:

```
cd ~/Labs
git clone -b hardware https://github.com/ENEE467-F2026/lab-2.git lab-2-hw
cd lab-2-hw/docker
```

Launch the lab-2 Docker container, as in Part 1:

```
userid=$(id -u) groupid=$(id -g) docker compose -f lab-2-compose.yml run \
--rm lab-2-docker
```

You should be greeted with the container prompt:

```
(lab-2) robot@docker-desktop:~$
```

The container already has the Robotics Toolbox for Python and the `ur_rtde` interface installed.

1.3 Hardware Setup and Verification



Figure 1: UR3e Teach Pendant. Emergency Stop button in red.

Before running any code, verify that the arm is powered and reachable:

- Power on the TP using the button beside the red emergency stop. Once the PolyScope GUI loads, the robot will be in a powered-off state. To enable it, click the bottom-left red button (next to **Power off**), then press ON to switch the robot to the **IDLE** state.
- Verify that the robot is reachable over Ethernet from your container session:

```
ping 192.168.77.22
```

You should see replies indicating successful communication:

```
64 bytes from 192.168.77.22: icmp_seq=49 ttl=64 time=0.329 ms
```

 The `ur_rtde` receive interface reads the robot's published state and does not require the Teach Pendant to be in **Play** mode or any external-control program to be loaded. Keep the robot in the **IDLE** state for this part.

2. Reading the Robot's Live State

The Universal Robots controller continuously publishes the robot's state over the Real-Time Data Exchange (RTDE) protocol. The `ur_rtde` library exposes this through a receive interface. Open the file `src/ur3_hw_validate.py`. Its first block connects to the robot and reads its state:

```
import roboticstoolbox as rtb
import numpy as np
import spatialmath as sm
from rtde_receive import RTDEReceiveInterface

ROBOT_IP = "192.168.77.22"

# Read-only connection to the UR3e. No motion commands are sent.
rtde_r = RTDEReceiveInterface(ROBOT_IP)
q = np.array(rtde_r.getActualQ())          # measured joint angles (rad)
tcp = np.array(rtde_r.getActualTCPPOSE()) # measured TCP pose [x, y, z, rx, ry, rz]

print("measured joint angles (rad):", np.round(q, 4))
print("measured TCP pose:", np.round(tcp, 4))
```

The call `getActualTCPPOSE` returns a length-six vector. The first three entries are the tool-center-point position in meters, expressed in the robot base frame. The last three entries (r_x, r_y, r_z) form a rotation vector (axis-angle): its direction is the axis of rotation and its magnitude is the rotation angle in radians.

Run the script (with the robot powered on and in the **IDLE** state):

```
cd ~/lab-2/src
python ur3_hw_validate.py
```

Question 1. Record the joint angles and TCP pose your robot reports. In which frame is the

TCP position expressed, and what does the rotation-vector portion of the pose encode? How would you convert this rotation vector into a rotation matrix?

3. Validating Forward Kinematics

We now compute the tool pose predicted by the kinematic model from the measured joint angles and compare it against the pose the robot reports. The next block of `ur3_hw_validate.py` loads the UR3 model and computes forward kinematics:

```
robot = rtb.models.DH.UR3()

# Forward kinematics from the measured joint angles
T_fk = robot.fkine(q)

# Rebuild the measured TCP pose as an SE(3): position and rotation vector
p = tcp[:3]
rvec = tcp[3:]
theta = np.linalg.norm(rvec)
axis = rvec / theta if theta > 1e-9 else np.array([0.0, 0.0, 1.0])
T_meas = sm.SE3(p) * sm.SE3.AngleAxis(theta, axis)

# Compare model-based and measured tool poses
pos_err = np.linalg.norm(T_fk.t - T_meas.t)
print("FK position error (m):", pos_err)
```

Question 2. Report the forward-kinematics position error `pos_err` for your measured configuration. The error is usually small but nonzero. Give two reasons a model-based forward-kinematics pose can disagree with the pose a real robot reports. (**Hint:** consider the difference between the nominal DH parameters and the robot's factory calibration, and the tool-center-point offset configured on the pendant.)

4. Validating Inverse Kinematics

Finally, we feed the measured tool pose back through inverse kinematics and check that the recovered configuration reproduces the pose. The last block computes IK with the Levenberg-Marquardt solver:

```
sol = robot.ikine_LM(T_meas)
print("IK converged:", sol.success)

T_ik = robot.fkine(sol.q)
print("IK pose residual (m):", np.linalg.norm(T_ik.t - T_meas.t))
print("measured q:", np.round(q, 4))
print("IK q:      ", np.round(sol.q, 4))
```

Question 3. Did the solver converge (`sol.success`)? Compare the recovered joint vector `sol.q` against the measured joint vector `q`. They often differ even when the pose residual is tiny. Explain why, referring to the multiplicity of inverse-kinematics solutions for a six-axis arm.

Question 4. The procedure in this part is deliberately read-only: it reads state and computes with it but never commands motion. Explain why validating kinematics against logged state is safe, and name two safeguards that would be required before closing the loop and commanding the arm to a model-computed configuration.

5. Exercises

Exercise 1. Forward-Kinematics Validation Across Configurations

Using the freedrive feature on the Teach Pendant (or by jogging the arm slowly under supervision), move the UR3e by hand to at least four distinct, safe configurations spread across its workspace. At each configuration, run `ur3_hw_validate.py` and record the measured joint vector, the measured TCP position, and the forward-kinematics position error. Present your results in a table with one row per configuration. Comment on whether the error is consistent across configurations or grows in particular regions of the workspace.

Deliverables: A table of at least four configurations listing the measured q , the measured TCP position, and the FK position error, plus one or two sentences interpreting the trend.

Exercise 2. Inverse-Kinematics Round-Trip

For each configuration you recorded in Exercise 1, feed the measured TCP pose through `ikine_LM` and verify that the forward kinematics of the recovered configuration reproduces the measured pose (report the pose residual). For each case, also report whether the recovered joint vector matches the measured one. Summarize, in a sentence or two, what your results show about the relationship between a tool pose and the joint configurations that achieve it.

Deliverables: For each configuration: the IK convergence flag, the FK pose residual of the IK solution, and whether `sol.q` matched the measured q , plus a short interpretation.

6. Conclusion and Lab Report Instructions

After completing the lab, summarize the procedure and results into a well written lab report. Include your solutions to all of the question boxes in the lab report. Refer to the [List of Questions](#) to ensure you don't miss any. Include your full solution to the exercises in your lab report. If you wrote any Python programs during the exercises, include these files as a separate attachment, appending your team's number and last names (in alphabetical order) to the file, e.g., **Jane Doe, Alice Smith, and Richard Roe** in **Team 90** submitting their modified `main.py` would submit the file `main_Team90_Doe-Roe-Smith.py` and the report `Lab-<LabNumber>-Report_Team90_Doe-Roe-Smith.pdf`, where `<LabNumber>` is the number for that week's lab, e.g., **0, 1, 2**, etc. Submit your lab report along with any code to ELMS under the assignment for that week's lab **by the deadline stated on the ELMS assignment**. See the **Files** section on ELMS for a lab report template named `Lab_Report_Template.pdf`. The grading rubric is as follows:

Component	Points
Lab Report and Formatting	10
Question 1.	5
Question 2.	5
Question 3.	5
Question 4.	5
Exercise 1.	35
Exercise 2.	35
Total	100